Rectificar o adaptar: un estudio factorial de la detección de objetos en cámaras *fisheye* automotrices

Franco Marcelo Morero, Nicolás Granato, Marcos Levi Riveros Koloszwa y Tadeo Riveros Koloszwa
Carrera de Especialización en Inteligencia Artificial (CEIA), Facultad de Ingeniería, Universidad de Buenos Aires
Visión por Computadora II – Grupo 1

***Resumen*—Las cámaras de visión envolvente de los autos usan lentes *fisheye* de casi 190 ° que deforman los objetos según su ángulo de incidencia θ. Un detector pre-entrenado en imágenes de perspectiva pierde allí casi todo su desempeño, justo en la zona de puntos ciegos. Comparamos las dos estrategias habituales para adaptarlo, rectificar la imagen con una proyección cilíndrica y hacer *fine-tuning* sobre las imágenes *fisheye*, en un diseño factorial 2×2 que aísla el efecto de cada una y su interacción, con el mismo YOLO11m, los mismos datos de WoodScape y la misma métrica: mAP estratificado por θ con el esquema de COCO, sobre 1000 imágenes de prueba sin fuga de información. Sin adaptar, el detector pierde el 69 % de su mAP@50 entre el centro y la periferia ($\theta > 80{,}7\,°$). El *fine-tuning* sin rectificar lleva la periferia de 0,170 a 0,603 y deja la caída en 14 %, sin costo de latencia. Rectificar mejora al detector sin entrenar (+0,147 en la periferia) pero perjudica al entrenado ($-0{,}191$): la interacción es de $-0{,}337$. La causa es la vuelta de las cajas a la imagen original, que limita la IoU alcanzable y hunde el mAP@50-95. Una augmentación de distorsión radial no mejora con las mismas lentes, pero reduce a la mitad la degradación frente a lentes distintas.**

***Palabras clave*—cámaras *fisheye*, detección de objetos, *fine-tuning*, proyección cilíndrica, WoodScape, YOLO11**

I. Introducción

Los sistemas de visión envolvente (*surround view*) combinan cuatro cámaras *fisheye* (frontal, trasera y una en cada espejo) que cubren 360 ° alrededor del vehículo y alimentan el estacionamiento automático, la detección de peatones a baja velocidad y el monitoreo de puntos ciegos [2]. Su lente sigue un modelo de proyección $\rho(\theta) = k_1\theta + k_2\theta^2 + k_3\theta^3 + k_4\theta^4$ [7], en el que la deformación de un objeto depende del ángulo de incidencia θ respecto del eje óptico: un peatón a 20 ° y el mismo peatón a 85 ° tienen geometrías muy distintas en la imagen. Los detectores disponibles se pre-entrenan en imágenes de perspectiva como COCO [8], de modo que la zona que más importa para la seguridad, la periferia, es justamente la que peor resuelven.

El problema ya llegó a la industria. Valeo, fabricante de estas cámaras, publicó WoodScape [1], el primer dataset público extenso de cámaras *fisheye* automotrices, y sobre él OmniDet (2021) [3], una red de percepción multitarea para las cuatro cámaras que corre en tiempo real en hardware embebido automotriz (NVIDIA Jetson AGX con TensorRT). Fuera del auto, FishEye8K (2023) [6] reúne 8000 imágenes de 18 cámaras *fisheye* de monitoreo de tránsito en intersecciones de Hsinchu, Taiwán, para detectar autos, buses, camiones, motos y peatones. En ambos casos el detector tiene que funcionar sobre la imagen deformada.

Hay dos familias de soluciones. Una rectifica la imagen y usa el detector tal como está: Plaut et al. [5] muestran que una proyección cilíndrica permite aplicar detectores entrenados solo con imágenes de perspectiva. La otra adapta el modelo con datos *fisheye*, y Rashed et al. [4] señalan además que la caja rectangular representa mal a los objetos deformados. Para un equipo que parte de un detector pre-entrenado, la pregunta práctica es cuál conviene y si vale la pena combinarlas. Los trabajos previos suelen evaluar una sola estrategia, con modelos, datos o métricas distintos, lo que impide comparar.

Objetivo. Medir, con el mismo detector, los mismos datos y la misma métrica, cuánto aporta rectificar, cuánto aporta el *fine-tuning* y cómo interactúan, con foco en la periferia y en la latencia. Las contribuciones son:

- un diseño factorial 2×2 (rectificar sí/no $\times$ *fine-tuning* sí/no) que separa los dos efectos y estima su interacción;
- una métrica estratificada por θ con el esquema *ignore* de COCO, validada contra `pycocotools`, sobre una partición auditada contra casi-duplicados;
- evidencia de que, una vez que el modelo se adapta, rectificar resta, y una explicación cuantitativa: el techo de IoU de la reproyección de cajas;
- una augmentación de distorsión radial que no cambia el desempeño con las lentes del dataset pero reduce a la mitad la pérdida con otras lentes.

II. Metodología, diseño y desarrollo

II-A. Datos y partición

Usamos las 8234 imágenes públicas de WoodScape [1] (cámaras FV, MVL, MVR y RV, 1280×966, con 23 juegos de calibración intrínseca). Las 43 etiquetas de instancia se agrupan en tres clases: `vehicle` (car, van, bus, truck, trailer, caravan), `person` (person, rider) y `two_wheeler` (bicycle, motorcycle). La caja de referencia es la envolvente del polígono, y se descartan las menores a 300 px^2 o con un lado menor a 8 px.

WoodScape proviene de grabaciones continuas, así que un muestreo aleatorio podría repartir cuadros casi idénticos entre entrenamiento y prueba. Con *hashes* perceptuales de 256 bits

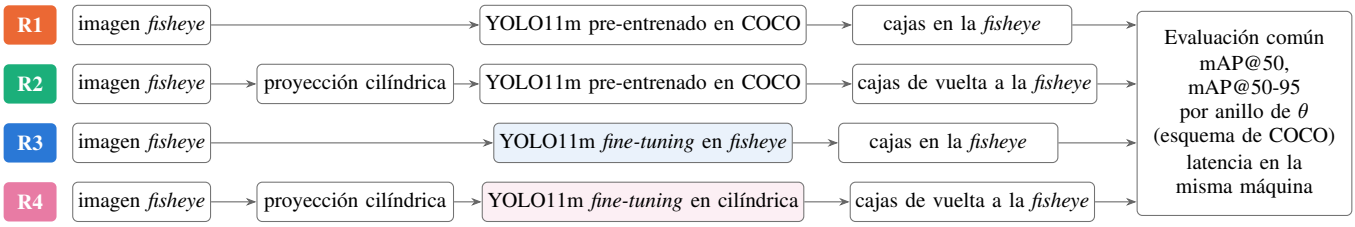


Figura 1. Diseño factorial 2×2. R1: el detector de COCO sobre la imagen original. R2: el mismo detector sobre la proyección cilíndrica, con las cajas llevadas de vuelta a la *fisheye*. R3: *fine-tuning* sobre la *fisheye*. R4: *fine-tuning* sobre la cilíndrica. Las cuatro se evalúan con la misma métrica sobre la imagen original.

verificamos que la distancia mediana entre cada imagen de prueba y su vecina más cercana de entrenamiento es de 39 bits (15,2 %); el único par sospechoso quedó en cuarentena. La partición final, estratificada por cámara, tiene 6148 imágenes de entrenamiento, 1085 de validación y 1000 de prueba con 11 910 objetos. Las imágenes se verifican por sha256 y nunca se recomprimen. La prueba se usó una sola vez, con los pesos finales.

II-B. Métrica estratificada por ángulo de incidencia

Para cada caja calculamos el θ de su centro invirtiendo numéricamente $\rho(\theta)$ con la calibración de su imagen. Estratificar por θ y no por radio en píxeles tiene dos ventajas: es la magnitud física que gobierna la deformación y es comparable entre cámaras con calibraciones distintas. Los cortes de los anillos se fijaron en los terciles de la distribución de la prueba, 68,3 ° y 80,7 °, de modo que centro, medio y periferia tienen 3970 objetos cada uno, y quedaron congelados para todas las ramas.

Dentro de cada anillo aplicamos el esquema `areaRng` de COCO con θ en lugar del área: los objetos de referencia de otros anillos se marcan *ignore*, una detección que coincide con uno de ellos no cuenta, y las detecciones sin pareja fuera del anillo se descartan. Reportamos mAP@50 y mAP@50-95 (101 puntos de recall, 100 detecciones por imagen y clase, umbral de confianza 0,01). Nuestra implementación coincide con `pycocotools` en todos los anillos (diferencia máxima 10^{-16}) y con los valores reportados para la rama 4 (diferencias $\leq$ 0,001). La latencia se mide por imagen en la misma máquina para las cuatro ramas (Apple M4 Max, MPS), con diez pasadas de calentamiento y sin contar la lectura del disco.

II-C. Las cuatro ramas

Las cuatro ramas (Fig. 1) usan YOLO11m [9] (20,1 M parámetros, 68,1 GFLOPs) con entrada de 640 px. Lo elegimos en el codo de precisión contra latencia de un barrido de ocho detectores pre-entrenados (YOLO11 n–x, Faster R-CNN y Mask R-CNN), y es un detector de una etapa apto para una ECU. Todas se evalúan sobre la imagen *fisheye* original, que es la que usa el auto.

R1, sin adaptar. Inferencia directa, con las clases de COCO traducidas a las del trabajo (car, bus, truck $\rightarrow$ `vehicle`; person; bicycle, motorcycle $\rightarrow$ `two_wheeler`).

R2, rectificar. Proyección cilíndrica de 190 ° horizontales [5], recortada a una banda de ± 45 ° alrededor del horizonte, que contiene el centro del 95,7 % de las cajas de prueba. No usamos la proyección de perspectiva porque diverge cerca de 90 ° y obligaría a recortar justamente la periferia. Cada caja detectada se lleva de vuelta muestreando 20 puntos por lado y tomando la envolvente del contorno curvo resultante.

R3, adaptar. *Fine-tuning* de YOLO11m desde los pesos de COCO sobre las imágenes *fisheye* sin modificar, con la cabeza reiniciada para las tres clases.

R4, ambas. *Fine-tuning* sobre la proyección cilíndrica de R2. Las etiquetas se obtienen proyectando el polígono de cada objeto y no su caja, que en la periferia quedaría inflada.

R3 y R4 comparten el entrenamiento: 30 épocas, augmentaciones por defecto de Ultralytics (mosaico, HSV, espejado y escala; sin mosaico en las 10 finales), lote nominal de 64 imágenes por acumulación de gradiente, AdamW con tasa $1{,}43 \times 10^{-3}$ y el *checkpoint* de mejor mAP@50-95 en validación. Solo difieren el lote físico (16 en R3, 2 en R4, por la memoria de su GPU) y la semilla. R3 se entrenó en una Apple M4 Max (3,4 h) y R4 en una GTX 1660 de 6 GB.

II-D. Extensión: augmentación de distorsión radial

Para que el modelo de R3 no dependa de las lentes del dataset agregamos una distorsión radial aleatoria, el primer término del modelo de Brown–Conrady [10]: cada píxel a radio normalizado r del centro óptico toma el valor del radio $r_{\text{src}} = r\,(1+kr^2)/(1+k)$, con $r = 1$ en la esquina más lejana, $k \sim \mathcal{U}[-0{,}2;\ 0{,}3]$ y probabilidad 0,5, aplicada a cada imagen antes del mosaico. Las cajas se transforman muestreando 16 puntos por lado. La transformación es monótona para $k > -1/3$. Esta variante se entrenó con los mismos parámetros en una GPU A100.

III. Resultados experimentales y análisis

III-A. Resultados por anillo

La Tabla I y la Fig. 2 resumen el diseño. Sin adaptar (R1), el detector pierde el 69 % de su mAP@50 del centro a la periferia. No es un problema del detector elegido: con el mismo evaluador, YOLOv8m queda en 0,164 en la periferia, y con el esquema de filtrado YOLO26m da 0,171 contra 0,160 de YOLO11m. Tres generaciones de YOLO quedan en el mismo lugar.

La caída es geométrica y no un efecto de escala. Separando por los rangos de área de COCO (Tabla II), en R1 los objetos grandes pasan de 0,841 en el centro a 0,510 en la

Tabla I
RESULTADOS EN LAS 1000 IMÁGENES DE PRUEBA (MAP POR ANILLO DE θ, ESQUEMA DE COCO). LA CAÍDA ES LA DEL MAP@50 DEL CENTRO A LA PERIFERIA; LA LATENCIA, LA MEDIA POR IMAGEN EN LA MISMA MÁQUINA.

	Global		mAP@50 por anillo				
Rama	mAP@50	50-95	centro	medio	perif.	caída	ms
R1 sin adaptar	0,378	0,239	0,541	0,432	0,170	69 %	25,0
R2 cilíndrica	0,418	0,155	0,528	0,420	0,316	40 %	26,6
R3 *fine-tuning*	**0,659**	**0,430**	**0,701**	**0,670**	**0,603**	**14 %**	**22,9**
R4 ambas	0,522	0,172	0,642	0,522	0,412	36 %	28,6

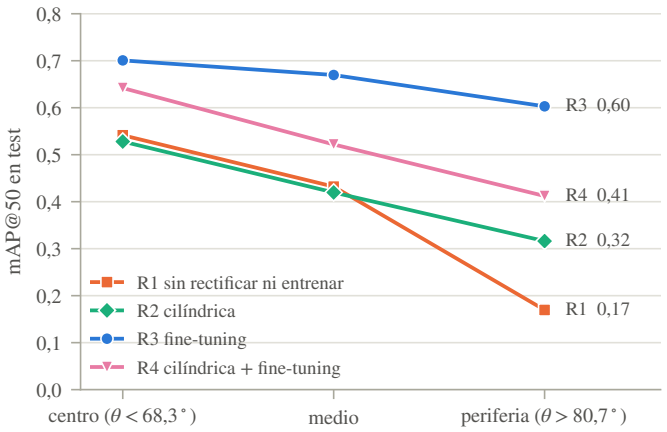


Figura 2. mAP@50 por anillo de ángulo de incidencia en la prueba. La rama de *fine-tuning* sin rectificar (R3) es la única que se mantiene casi plana hacia la periferia.

periferia (-39 %) y los chicos de 0,301 a 0,030 (-90 %): la deformación es más destructiva cuantos menos píxeles describen al objeto. El *fine-tuning* recupera los tres tamaños, y en la periferia multiplica por diez el mAP@50 de los objetos chicos (0,030 a 0,296).

III-B. Efectos del diseño factorial

Cada efecto de la Tabla III es la diferencia entre dos ramas que cambian un solo factor. El *fine-tuning* es el factor dominante: sin rectificar, suma 0,433 de mAP@50 en la periferia. Rectificar ayuda al detector sin entrenar (+0,147 en la periferia), pero al detector entrenado le resta 0,191. La interacción, $(\mathrm{R4}-\mathrm{R3})-(\mathrm{R2}-\mathrm{R1})$, es de $-0{,}337$ en la periferia. Partimos de la hipótesis de que rectificar aportaría menos una vez que el modelo vio datos *fisheye*; los datos muestran algo más fuerte: aporta en contra. La conclusión no depende del criterio de anillos: con el esquema de filtrado la interacción es de $-0{,}339$ y con anillos por radio en píxeles, de $-0{,}354$.

III-C. Por qué rectificar resta

Las ramas rectificadas pierden sobre todo en mAP@50-95: 0,155 y 0,172, contra 0,239 de R1 y 0,430 de R3. La causa está en la vuelta de las cajas. Una caja recta en la cilíndrica es un contorno curvo en la *fisheye*, y su envolvente queda más grande que el objeto. Si se proyectan las cajas de referencia

Tabla II
MAP@50 POR ANILLO Y TAMAÑO DE OBJETO (RANGOS DE ÁREA DE COCO: CHICO $< 32^2$, MEDIANO $< 96^2$ PX). ENTRE PARÉNTESIS, LA CANTIDAD DE OBJETOS.

	Centro			Periferia		
Rama	chico (1546)	mediano (1790)	grande (634)	chico (1288)	mediano (2147)	grande (535)
R1	0,301	0,654	0,841	0,030	0,175	0,510
R2	0,334	0,636	0,786	0,092	0,392	0,639
R3	**0,472**	**0,806**	**0,946**	**0,296**	**0,695**	**0,892**
R4	0,451	0,747	0,885	0,191	0,480	0,739

Tabla III
EFECTOS DEL DISEÑO 2×2 EN LA PRUEBA (DIFERENCIAS DE MAP, ANILLOS POR θ).

Efecto	mAP@50 global	mAP@50 perif.	mAP@50-95 perif.
Rectificar sin *fine-tuning* (R2−R1)	+0,041	+0,147	+0,010
Rectificar con *fine-tuning* (R4−R3)	−0,138	−0,191	−0,252
Fine-tuning sin rectificar (R3−R1)	+0,282	+0,433	+0,278
Fine-tuning con rectificar (R4−R2)	+0,104	+0,096	+0,016
Interacción (R4−R3)−(R2−R1)	**−0,178**	**−0,337**	**−0,261**

y se las lleva de vuelta, es decir con una detección perfecta, la IoU mediana con la caja original es de 0,81 en el centro de la imagen, pero de 0,65 y 0,66 en los anillos medio y exterior (en este control los anillos son por radio en píxeles). Además, el 6,9 % y el 11,6 % de esos objetos no llegan a IoU 0,5, y el 3,4 % queda fuera de la banda y es irrecuperable. No es que R4 entrene peor: en su propia validación cilíndrica alcanza un mAP@50-95 de 0,446, a la par del 0,431 de R3 en la *fisheye*. La pérdida aparece al volver a la imagen original, y ningún entrenamiento la compensa. Con el detector sin entrenar, en cambio, la ganancia geométrica en la periferia supera ese costo a IoU 0,5.

III-D. Cámaras, clases y ejemplo cualitativo

Las cámaras laterales, que miran hacia abajo y ven casi toda la calle en la periferia, son las más afectadas (Tabla IV). R3 las lleva de 0,26 a 0,60–0,65 de mAP@50, por encima de lo que R1 logra en la frontal. Por clase, en la periferia `vehicle` pasa de 0,185 a 0,816, `person` de 0,207 a 0,507 y `two_wheeler` de 0,117 a 0,486. La Fig. 3 muestra el caso típico: el detector de COCO no encuentra los autos estirados del borde de una cámara lateral, que el modelo adaptado detecta.

III-E. Latencia

Con las cuatro ramas en la misma máquina y en la misma sesión, el detector tarda entre 23 y 25 ms por imagen en todas (Tabla I), una diferencia del orden de la variación entre corridas, porque es el mismo modelo. La proyección cilíndrica y la vuelta de las cajas suman unos 3 ms (1,6–1,7 ms y 1,3–1,5 ms), alrededor de un 12 % más. El *fine-tuning* no agrega costo: son los mismos pesos con otra cabeza de tres clases. Los valores absolutos cambian entre sesiones (17 ms en otra corrida del mismo modelo); la diferencia entre ramas no.



Figura 3. Cámara lateral derecha (imagen 02794_MVR de la prueba). Izquierda: referencia. Centro: R1 solo detecta las bicicletas del borde superior. Derecha: R3 encuentra los autos estirados de la periferia izquierda y derecha (umbral de visualización 0,3).

Tabla IV
MAP@50 GLOBAL POR CÁMARA EN LA PRUEBA.

Rama	FV (frontal)	MVL (izq.)	MVR (der.)	RV (trasera)
R1	0,521	0,259	0,264	0,495
R2	0,498	0,354	0,363	0,474
R3	**0,701**	**0,603**	**0,653**	**0,671**
R4	0,625	0,453	0,468	0,547

III-F. Entrenamiento y augmentación de distorsión radial

En R3, el mAP@50-95 de validación sube 0,123, 0,062 y 0,026 en cada tramo de diez épocas, con rendimientos decrecientes y sin sobreajuste: validación y prueba dan 0,652 y 0,659 de mAP@50. Más épocas sumarían del orden de centésimas, frente a una ganancia de 0,43 en la periferia.

Con las lentes del dataset, la augmentación radial no cambia el resultado (0,656 y 0,426 de mAP@50 y mAP@50-95 en la prueba, contra 0,659 y 0,430). Su aporte aparece cuando la lente cambia (Fig. 4). Distorsionando 300 imágenes de validación con k entre $-0{,}2$ y $+0{,}3$, el modelo base pierde hasta 0,063 de mAP@50-95 y el entrenado con la augmentación, 0,027. La pérdida se reduce a la mitad sin costo en las lentes conocidas. Como esta variante se entrenó en otra plataforma, solo atribuimos a la augmentación la diferencia de robustez, no las diferencias de milésimas en la prueba.

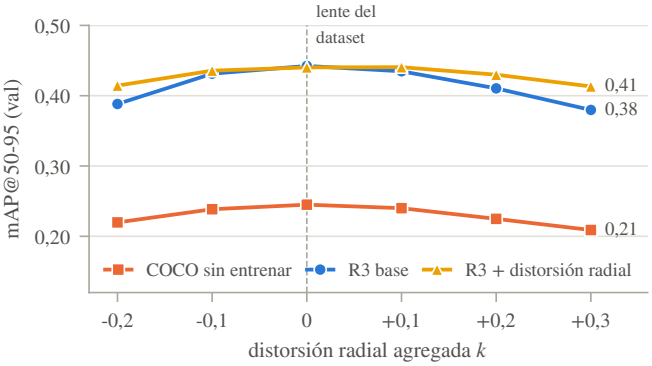


Figura 4. Robustez a una distorsión radial agregada en validación (300 imágenes). En $k = 0$ las tres ven la lente del dataset; a la derecha la lente es más *fisheye*, a la izquierda menos.

IV. CONCLUSIONES

1. **La distorsión es el problema, no el detector.** Un YOLO11m de COCO pierde el 69 % de su mAP@50 del centro a la periferia, la pérdida persiste dentro de cada rango de tamaño, y tres generaciones de YOLO quedan entre 0,16 y 0,17 en la periferia.
2. **Adaptar el modelo es la mejor estrategia.** El *fine-tuning* sobre la imagen *fisheye* lleva la periferia de 0,170 a 0,603 y el mAP@50 global de 0,378 a 0,659, deja la caída en 14 %, mejora las cuatro cámaras y no agrega latencia: para una ECU es el mismo modelo con otros pesos.
3. **Rectificar solo sirve sin entrenar.** La proyección cilíndrica mejora al detector de COCO en la periferia (+0,147), pero con *fine-tuning* le resta en la periferia 0,191 de mAP@50 y 0,252 de mAP@50-95. La interacción, de $-0{,}337$, es más fuerte que la hipótesis inicial.
4. **El costo está en la representación con cajas.** Llevar cajas de la cilíndrica a la *fisheye* limita la IoU alcanzable (mediana de 0,65–0,66 fuera del centro, aun con detección perfecta) y deja objetos fuera de la banda. Todo esquema "rectificar, detectar y volver" paga ese costo, que ningún entrenamiento recupera.
5. **La augmentación radial da robustez, no precisión.** Con las mismas lentes no cambia el resultado, pero reduce a la mitad la degradación ante lentes distintas. Es útil para flotas con cámaras heterogéneas.
6. **La evaluación tiene que ser por θ y en el espacio nativo.** El mAP global oculta la periferia, y medir cada rama en su propio espacio habría ocultado el costo de rectificar. Las conclusiones se mantienen con tres criterios de anillo y con un evaluador validado contra `pycocotools`.

Limitaciones y trabajo futuro. Hicimos una corrida por rama, sin varianza entre semillas: diferencias de 0,01–0,02 no son significativas, pero los efectos principales están un orden de magnitud por encima. R3 y R4 difieren en el lote físico y la semilla. Con representaciones curvas [4], como polígonos o cajas rotadas, las ramas rectificadas podrían perder

menos. Quedan pendientes una máscara del vehículo propio, más semillas y la medición de latencia en hardware embebido.

REFERENCIAS

[1] S. Yogamani, C. Hughes, J. Horgan, G. Sistu, P. Varley, D. O'Dea, M. Uricar, S. Milz, M. Simon, K. Amende, C. Witt, H. Rashed, S. Chennupati, S. Nayak, S. Mansoor, X. Perrotton y P. Pérez, "WoodScape: A multi-task, multi-camera fisheye dataset for autonomous driving," en *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, doi: 10.1109/ICCV.2019.00940.

[2] V. R. Kumar, C. Eising, C. Witt y S. K. Yogamani, "Surround-view fisheye camera perception for automated driving: Overview, survey & challenges," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 3638–3659, 2023.

[3] V. R. Kumar, S. K. Yogamani, H. Rashed, G. Sistu, C. Witt, I. Leang, S. Milz y P. Mäder, "OmniDet: Surround view cameras based multi-task visual perception network for autonomous driving," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 2830–2837, 2021.

[4] H. Rashed, E. Mohamed, G. Sistu, V. R. Kumar, C. Eising, A. El-Sallab y S. Yogamani, "Generalized object detection on fisheye cameras for autonomous driving: Dataset, representations and baseline," en *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, 2021, pp. 2272–2280.

[5] E. Plaut, E. Ben-Yaacov y B. El Shlomo, "3D object detection from a single fisheye image without a single fisheye training image," en *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, 2021, pp. 3659–3667.

[6] M. Gochoo, M.-E. Otgonbold, E. Ganbold, J.-W. Hsieh, M.-C. Chang, P.-Y. Chen, B. Dorj, H. Al Jassmi, G. Batnasan, F. Alnajjar, M. Abduljabbar y F.-P. Lin, "FishEye8K: A benchmark and dataset for fisheye camera object detection," en *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, 2023, pp. 5305–5313.

[7] J. Kannala y S. S. Brandt, "A generic camera model and calibration method for conventional, wide-angle, and fish-eye lenses," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 28, no. 8, pp. 1335–1340, 2006.

[8] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár y C. L. Zitnick, "Microsoft COCO: Common objects in context," en *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2014, pp. 740–755.

[9] G. Jocher y J. Qiu, "Ultralytics YOLO11," versión 11.0.0, 2024. [En línea]. Disponible: https://github.com/ultralytics/ultralytics

[10] D. C. Brown, "Decentering distortion of lenses," *Photogramm. Eng.*, vol. 32, no. 3, pp. 444–462, 1966.